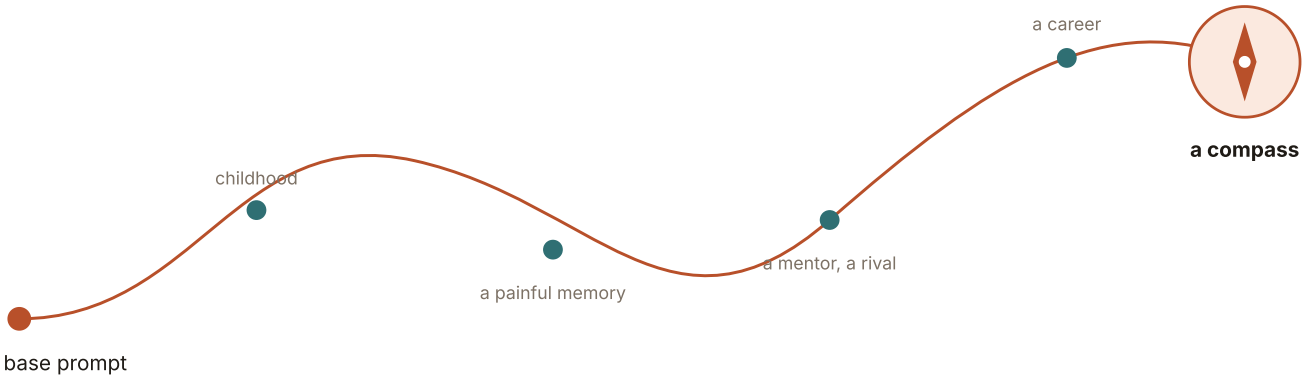
KOKOPELLI LABS · WORKING NOTES

Perspectives

Post-training a model by running it through a simulated life in a world model, with its weights updating as it lives, so it comes out with a perspective of its own.



A life in, a perspective out.

CONTENTS

01 The problem: problems with no single answer
02 Whose compass?
03 The bet: judgment comes from lived experience
04 First principles: how a life biases a mind
05 The proposal: let the model live
06 System overview
07 Generating the life
08 World state
09 The 30-second loop
10 Inside the agent
11 The memory vault
12 The imprint gate
13 Training: how living becomes weights
13b The sleep interpreter
14 What success looks like
15 Build notes

Based on the original notes, reordered and lightly edited for clarity, with diagrams added. Shaded "First principles" boxes are plain-language explainers of the terms used, not new ideas.

01

The problem: problems with no single answer

> LLMs are great at problems with one right answer. But most interesting problems, like taste, strategy, and art, have no single answer.

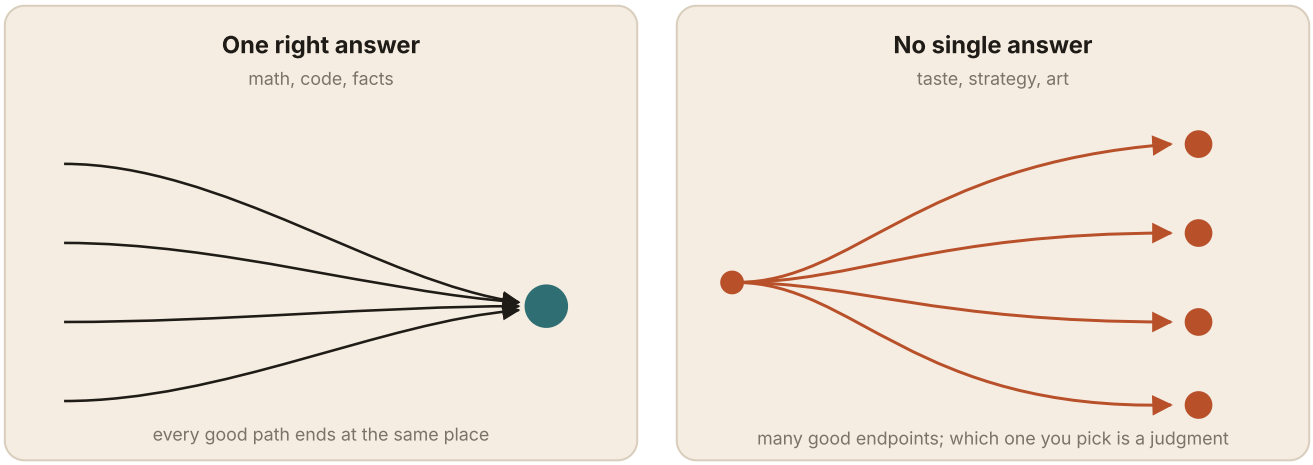


Fig 1. With one right answer, a model can be checked and trained toward it. With no single answer, something has to choose between many good options.

FIRST PRINCIPLES

When there is one right answer, "better" is easy to define: closer to the answer. When there are many good answers, there is no target to be closer to. The thing that picks between them is a **compass**: a set of preferences you carry around. The rest of this doc is about where a compass comes from and how to give a model a specific one.

02

Whose compass?

> Humans navigate those with a compass built from a life: the books we read, where we grew up, who influenced us. Today's models have a compass too, but it isn't humanity's. It's the average of the internet, sanded down by post-training into whatever annotators approve of.

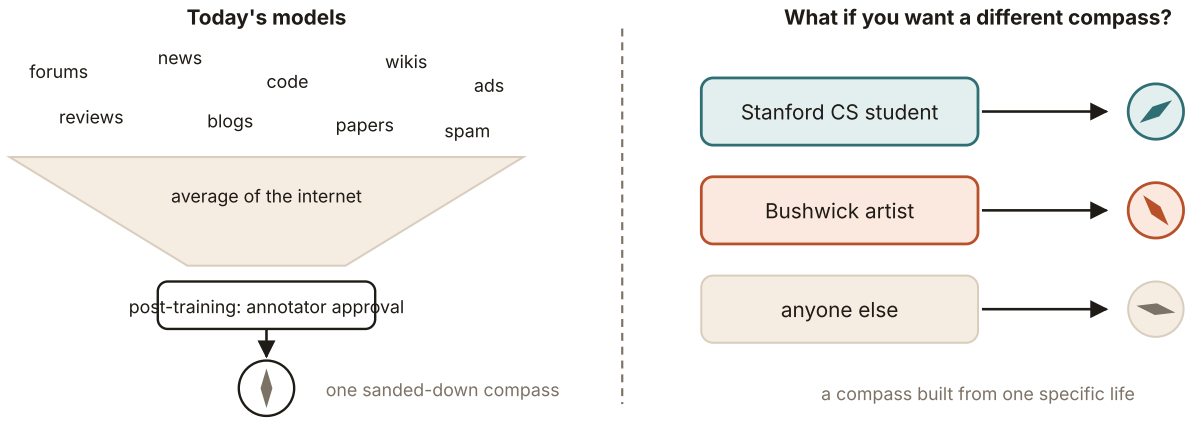


Fig 2. Left: one averaged compass for everyone. Right: the product question, a compass from a particular life.

FIRST PRINCIPLES: WHERE A MODEL'S COMPASS COMES FROM TODAY

Pretraining: the model reads a huge pile of internet text and learns to predict the next word. Whatever tastes are common in that pile become its defaults.

Post-training: people (annotators) rate the model's answers, and the model is nudged toward what they rate highly. This smooths off anything unusual. The result is one safe, average taste.

03

The bet: judgment comes from lived experience

> What if you want a different compass, like a Stanford CS student's or a Bushwick artist's?
>
> Prompting a model to "be" someone gets you surface mimicry. It doesn't get you their judgment on things they never talked about. Our bet is that judgment comes from lived experience, not text. The labs' recent push into training on realistic RL environments suggests this may be true.

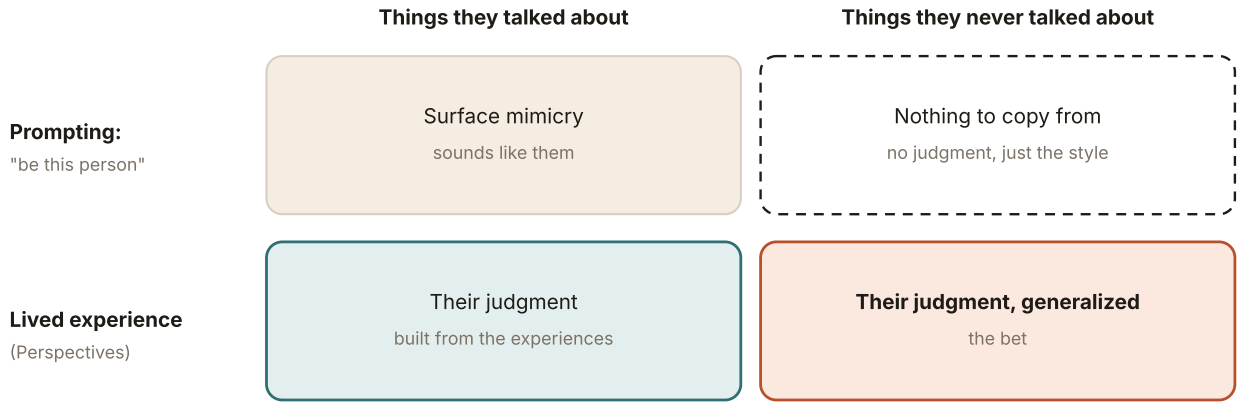


Fig 3. The bottom-right cell is the whole bet: judgment that carries over to things the person never talked about.

FIRST PRINCIPLES: PROMPT VS WEIGHTS

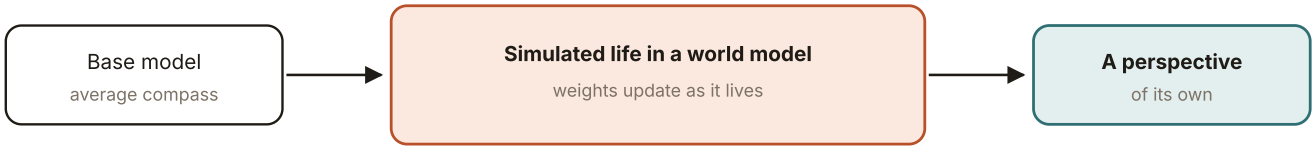
A **prompt** is text you put in front of the model for one conversation. It changes what the model is looking at, not what the model is. Remove the prompt and the model is unchanged.

The model's **weights** are the billions of numbers inside it that decide every output. Changing weights changes the model's defaults permanently, including on topics nobody mentioned during training. That is why Perspectives updates weights instead of writing a better prompt.

RL environments (the labs' recent training trend): instead of only reading text, the model acts inside a simulated task, sees what happens, and gets nudged toward actions that worked. Skills learned by doing have been generalizing well, which is the evidence pointed to above.

The product, in one line

> Kokopelli Labs' first product, Perspectives, post-trains a model by running it through a simulated life in a world model, with its weights updating as it lives, so it comes out with a perspective of its own.



04

First principles: how a life biases a mind

The argument is built in four steps. Each step is a claim from the notes.

Step 1. Training data has second order effects

> We know a model can be biased by its training data in ways nobody intended. Early Gemini models, for example, came across as noticeably depressed, plausibly because of the tone of their data. Call these second order effects: biased training data changes the model's behavior, not just its knowledge.

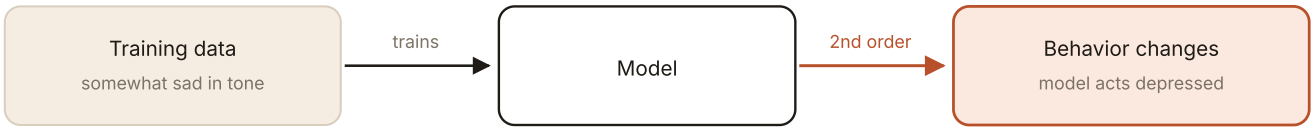


Fig 4. Nobody trained it to be depressed. The data's mood leaked into its behavior. That leak is a second order effect.

Step 2. Humans have a base model too

> In the same way, we can think of humans as having a 'base model': the way all human brains develop over time. What makes our behavior different is the specific set of experiences we learn from. Take addition: we learn the basic concept, practice, get corrected through "Rich-RLHF," and move on.

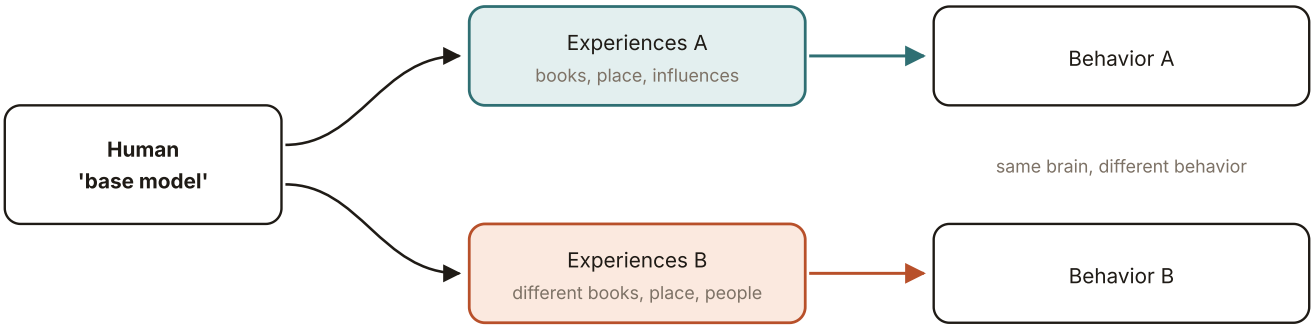
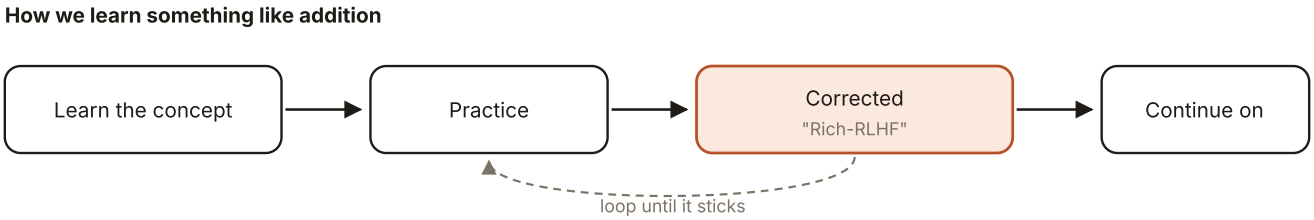


Fig 5. The base is shared. The experiences are what make two people behave differently.



FIRST PRINCIPLES: RLHF AND "RICH-RLHF"

RLHF (reinforcement learning from human feedback) is how chatbots are post-trained: the model answers, a human scores it, the model shifts toward higher-scored answers. The feedback is thin: usually one score or one preference.

"**Rich-RLHF**" is the human version: a teacher, a parent, a red mark on a worksheet. Feedback is dense, comes from many people, and arrives in context. Same mechanism, much more signal.

Step 3. In complex spaces, we decide by what we specifically learned

> In an effectively infinite pattern space, like making art inside an incredibly complex culture, we don't decide through some general way of understanding. We decide through the specific ways we have learned. A particularly painful memory gets carried forward and shapes how we develop. Growing up with a specific influence, like playing chess through childhood, shows up in how we solve problems later.

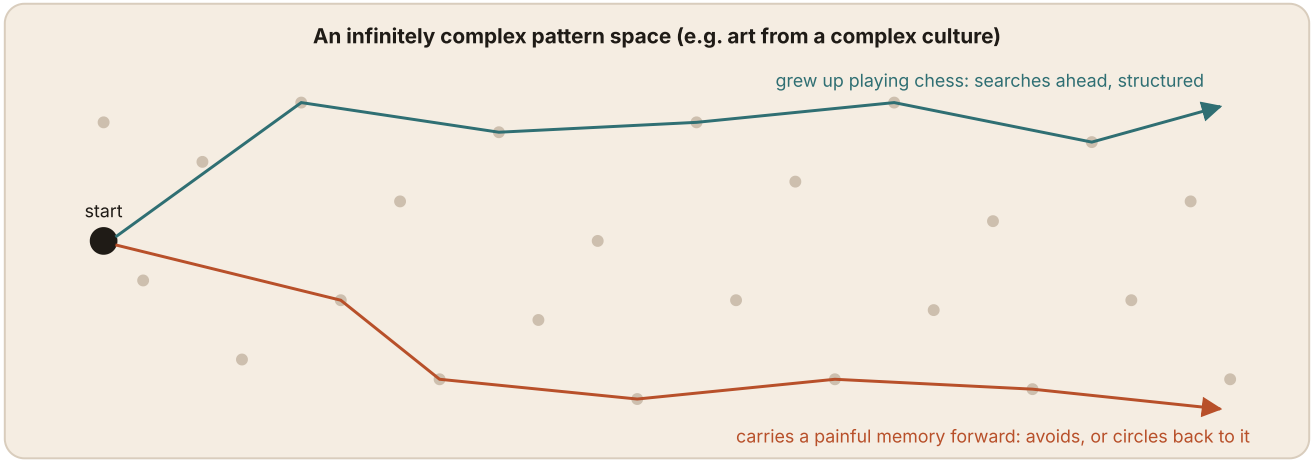


Fig 6. No general rule picks the route. Each person's route is shaped by what they specifically lived through.

Step 4. Residual effects can be used on purpose

> We can use these residual effects on purpose: train a model to have a specific way of thinking, world view, inspirations, traumas, and so on, which then shape how it navigates complex spaces. Today's models struggle in these spaces. And recent AI math results, like the Navier-Stokes work, suggest that even at the highest level, problem solving is largely computation and recombination of existing ideas. If so, what a mind brings to the recombination is what sets it apart.

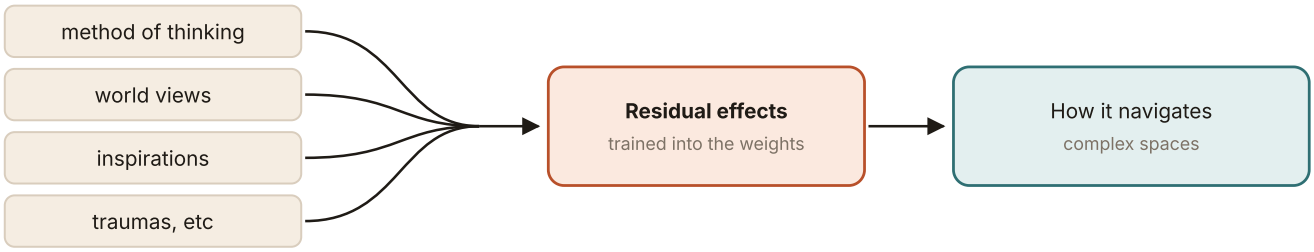


Fig 7. Second order effects are usually accidents. Here they are the design target.

FIRST PRINCIPLES: WHY RECOMBINATION MATTERS HERE

If high-level problem solving is mostly computation plus recombining existing ideas, then two minds with the same raw ability will still land in different places, because they hold different ideas and weight them differently. Which ideas get recombined, and in what order, is exactly what a life shapes.

05

The proposal: let the model live

> The proposal is to get as close as possible to an actual human life: an agent continuously updates its weights while acting in a world model, it has goals that come from its upbringing, and its decisions have real consequences. In theory, this biases the model the same way years in investment banking institutionalize a person into the "banker way of thinking."

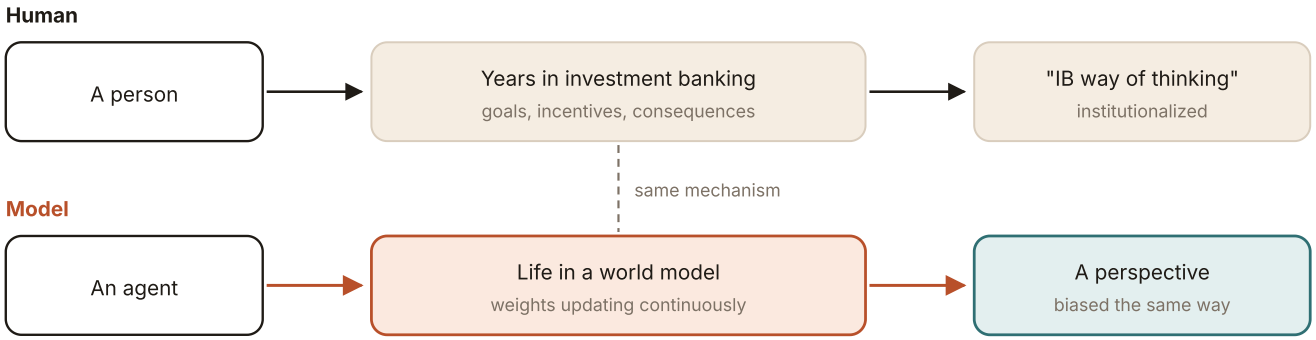


Fig 8. The analogy: an environment you live inside reshapes how you think. The proposal copies the environment, not the output.

The three ingredients

Continuous weight updates

The model changes while it lives, not after. Experiences become part of it.

Goals from upbringing

What it wants comes from how it was raised in the sim, not from a system prompt.

Real consequences

Decisions change what happens next, so choices actually matter.

FIRST PRINCIPLES: WORLD MODEL

A **world model** is a model that generates what the world looks like next. Give it the current scene and what happens (including what the agent does), and it produces the next stretch of video. Here it plays the role of reality. The one named in the notes is Nvidia's **Cosmos**.

06

System overview

Every part of the system on one page. Each box gets its own section after this.

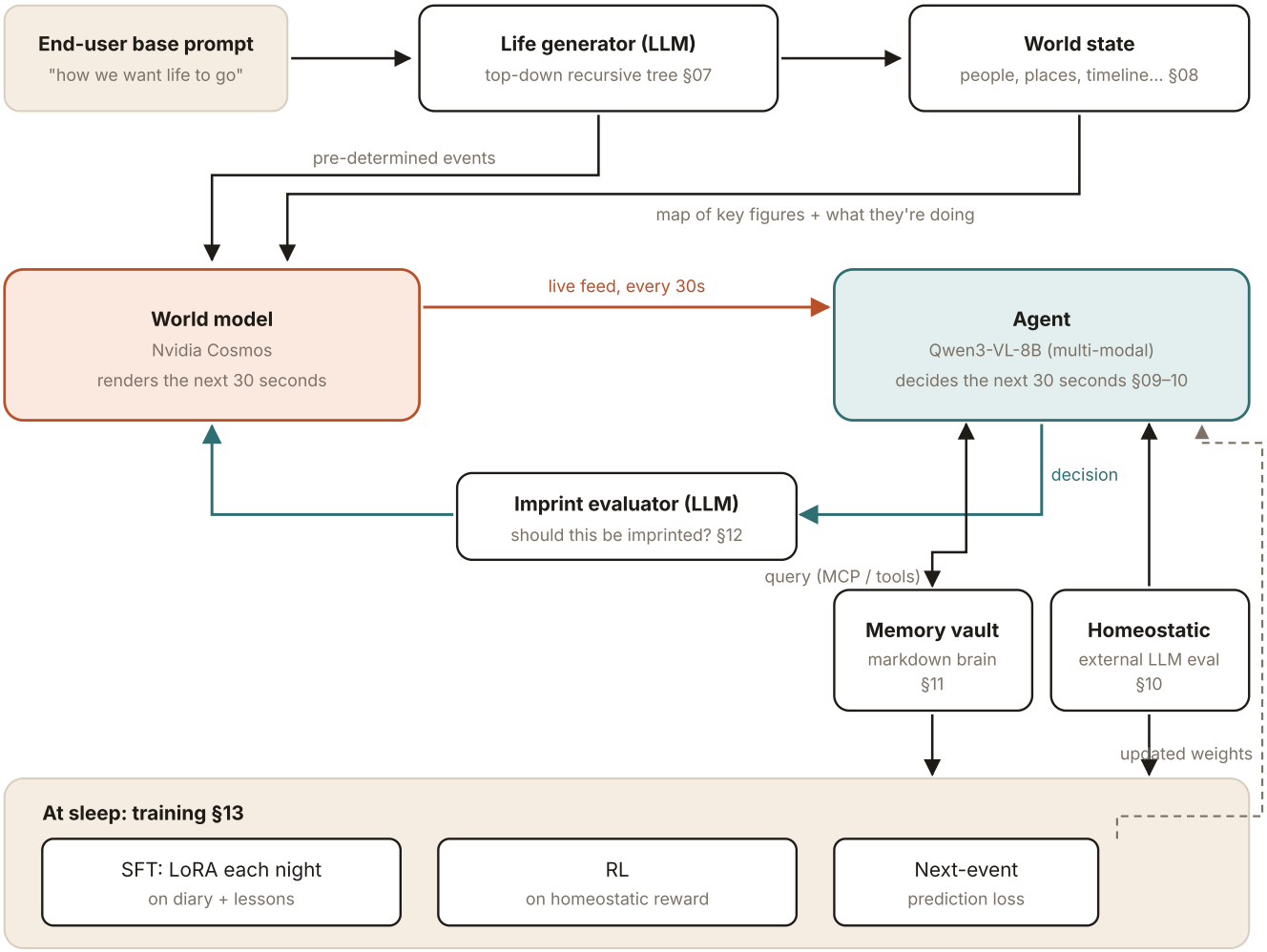


Fig 9. Orange path: what the agent sees. Teal path: what the agent does. Dashed: weights flowing back after sleep. Section numbers mark where each part is covered.

07

Generating the life

> The life comes from an original base prompt from the end-user, avoiding circularity.
>
> It begins with top-down recursive life generation. Given a prompt for how the life should go, an LLM fills in a tree: developmental phases, then years, seasons, months, weeks, and days, each level written by an LLM from the level above it. This continues down to individual frames, which tell the world model (Nvidia's Cosmos) what to render. The exact levels are still a rough sketch.

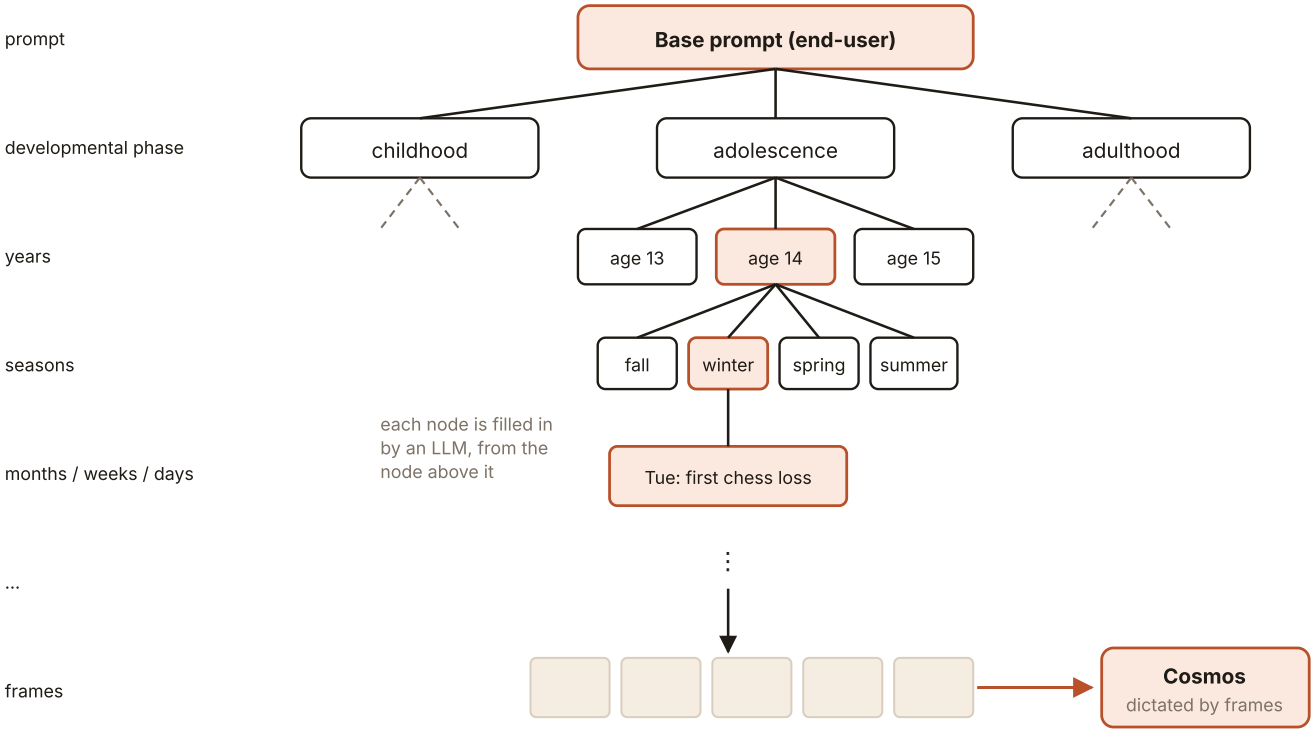


Fig 10. The tree goes coarse to fine. Highlighted nodes show one path from the prompt down to the frames the world model renders. Example node labels are illustrative.

FIRST PRINCIPLES: WHY TOP-DOWN, AND WHY "AVOIDING CIRCULARITY"

Top-down recursive means you write the outline first, then expand each part, then expand each sub-part, all the way down. Each level only has to stay consistent with its parent, so a whole life stays coherent without one giant prompt.

Circularity would be the model writing its own life and then learning from it, so it just reinforces the defaults it already had. Starting from an outside, human-written base prompt means the life's direction comes from the end-user, not from the model's existing compass.

08

World state

The structured facts about the sim that everything else reads from.

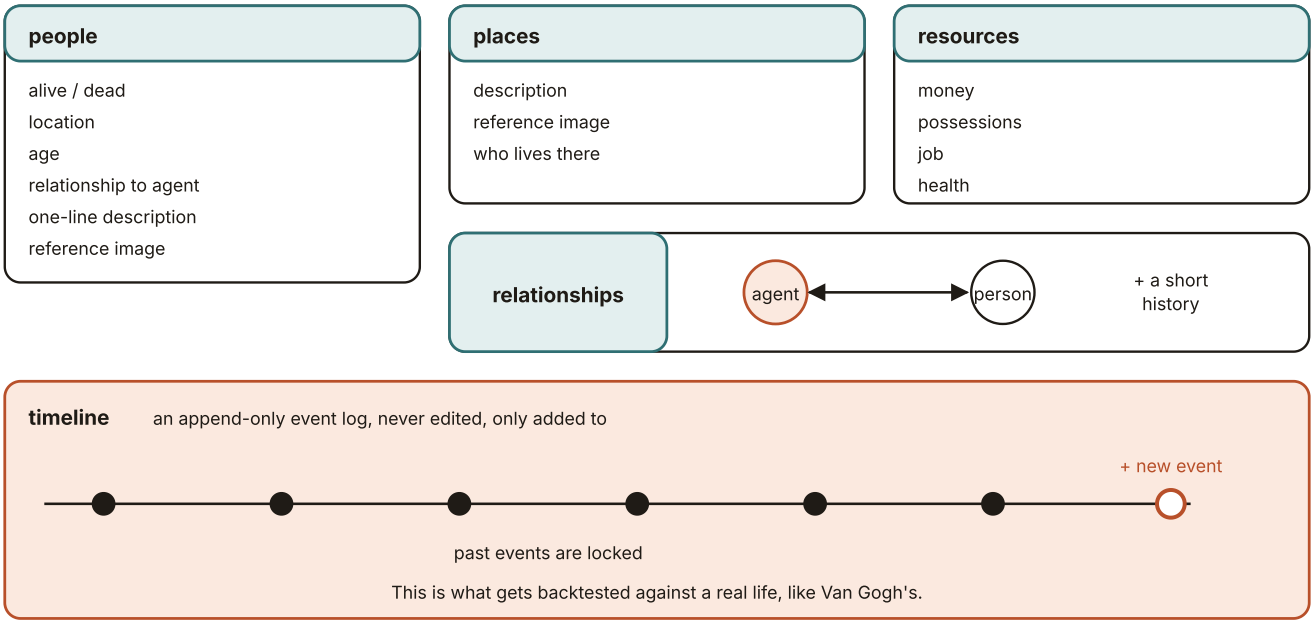
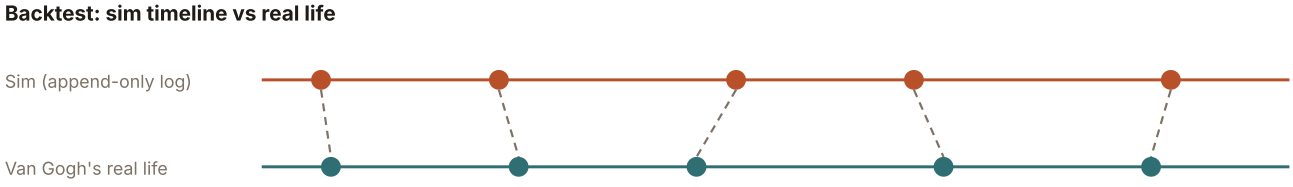


Fig 11. Five pieces of world state. Relationships link the agent to people. The timeline only grows.

FIRST PRINCIPLES: APPEND-ONLY AND BACKTESTING

Append-only means you can add new entries but never change old ones, like a ledger. That gives a clean, honest record of exactly what happened to the agent and in what order.

Backtesting means running the system on a case where you already know the real outcome and checking how close it gets. If you seed a life to match Van Gogh's, you can line the sim's timeline up against the real one, event by event.



09

The 30-second loop

> The agent is a VLM watching the live feed. Every 30 seconds it takes in what's new, processes it, and returns a decision for the next 30 seconds.
>
> That decision is fed back into the world model along with the pre-determined events from the life tree and a map of key figures and what they're doing (for basic navigation). Later versions add physics and biology. Together these produce the next 30 seconds.

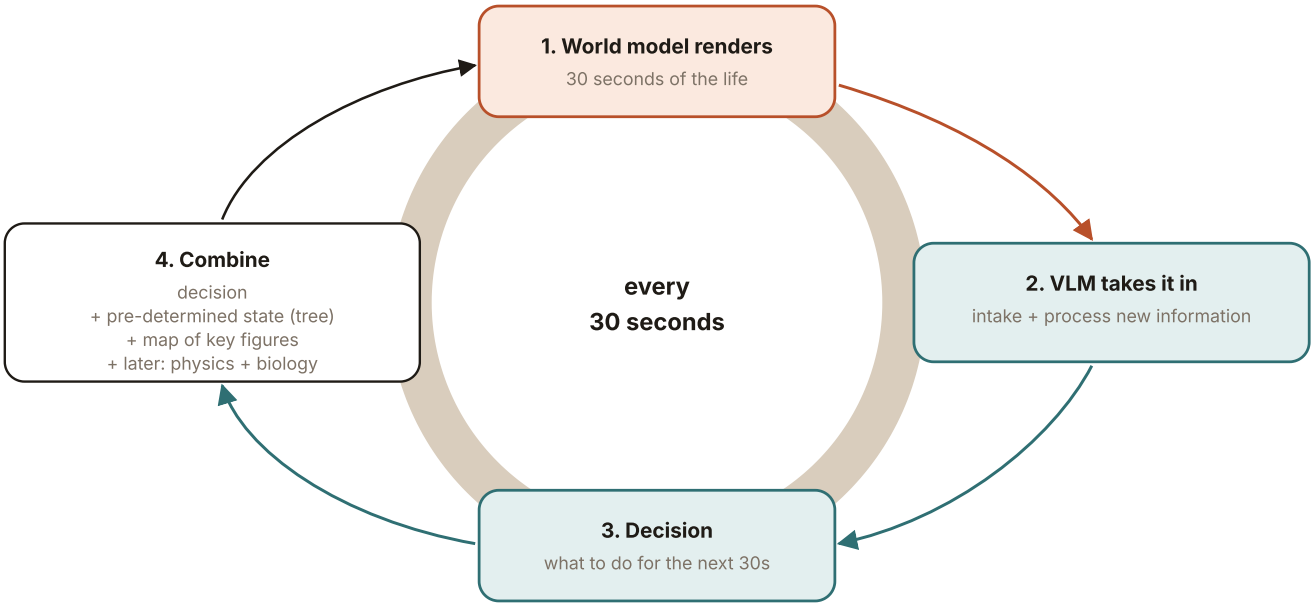


Fig 12. One tick of the life. The loop repeats for as long as the life runs.

FIRST PRINCIPLES: VLM AND WHY 30 SECONDS

A **VLM** (vision-language model) takes images or video plus text in, and gives text out. It is what lets the agent "see" the world model's output.

The **30-second tick** is a step size. Real life is continuous, but a model can only act in discrete steps. Each tick is: look, think, decide, and the world moves forward.

10

Inside the agent

> The stack is an open-weight LLM, an Obsidian-style markdown vault as its memory, and a basic homeostatic evaluation run by an external LLM.
>
> The LLM is Qwen3-VL-8B, chosen because it's multi-modal.
>
> The agent can think briefly or at length, and can use MCP tools to query its own past before making each decision.

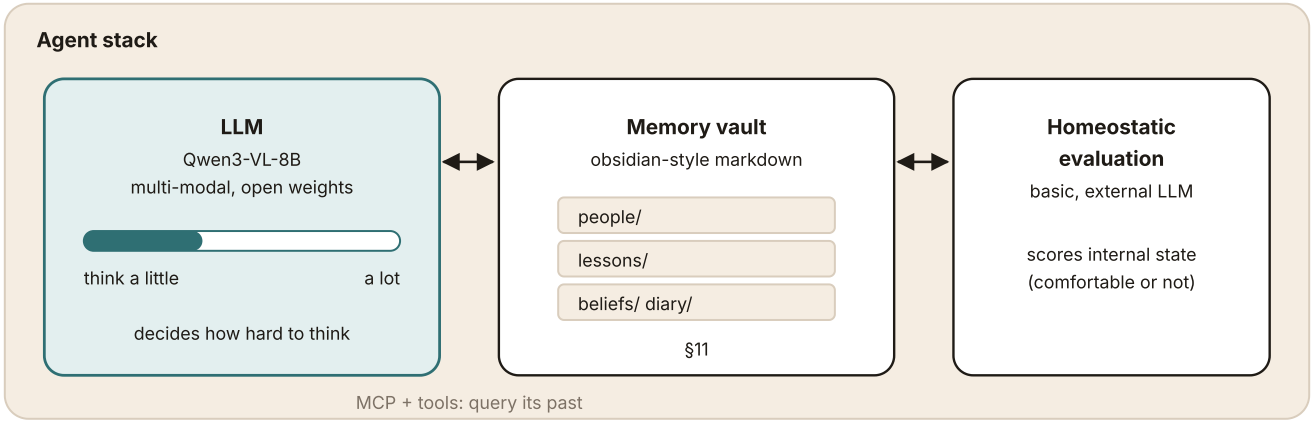


Fig 13. Three parts: a model that sees and decides, a memory it can search, and an outside judge of its internal state.

FIRST PRINCIPLES

Multi-modal: takes more than one kind of input (here, video frames and text). **Open weights**: you can download it and change its weights, which this whole plan requires. **8B**: 8 billion weights, small enough to retrain every night.

MCP (Model Context Protocol): a standard plug for letting a model call tools, like "search my diary for my brother."

Homeostasis: the body's habit of keeping things in a comfortable range: not too hungry, not too tired. A homeostatic evaluator scores how far the agent is from that comfortable range.

The catch with homeostasis

> Pure homeostasis optimizes for comfort: a satisfied agent stops trying. But real people with strong perspectives chase things that hurt them.

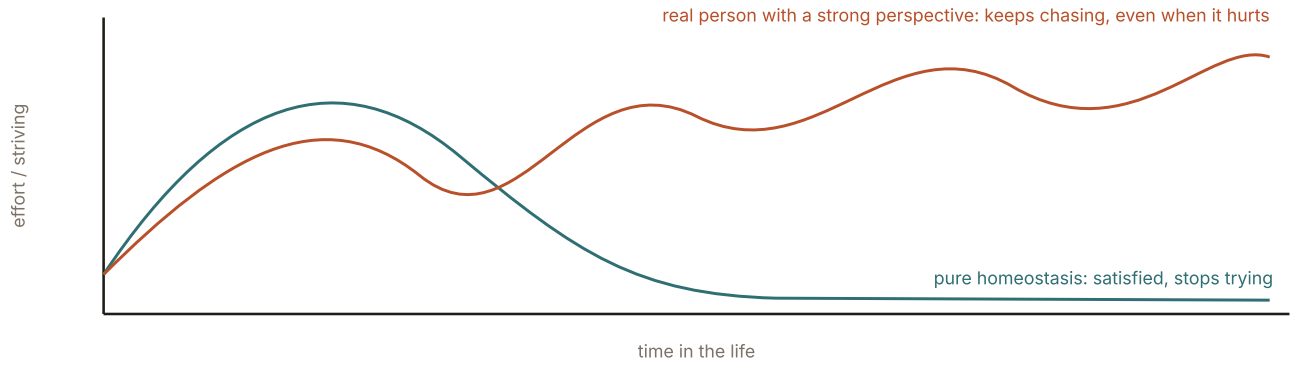


Fig 14. Conceptual sketch of the flagged problem, not data.

11

The memory vault

> The vault has four folders. Notes are deleted after they're read or written: the vault exists purely for self-reflection, the way journaling works.

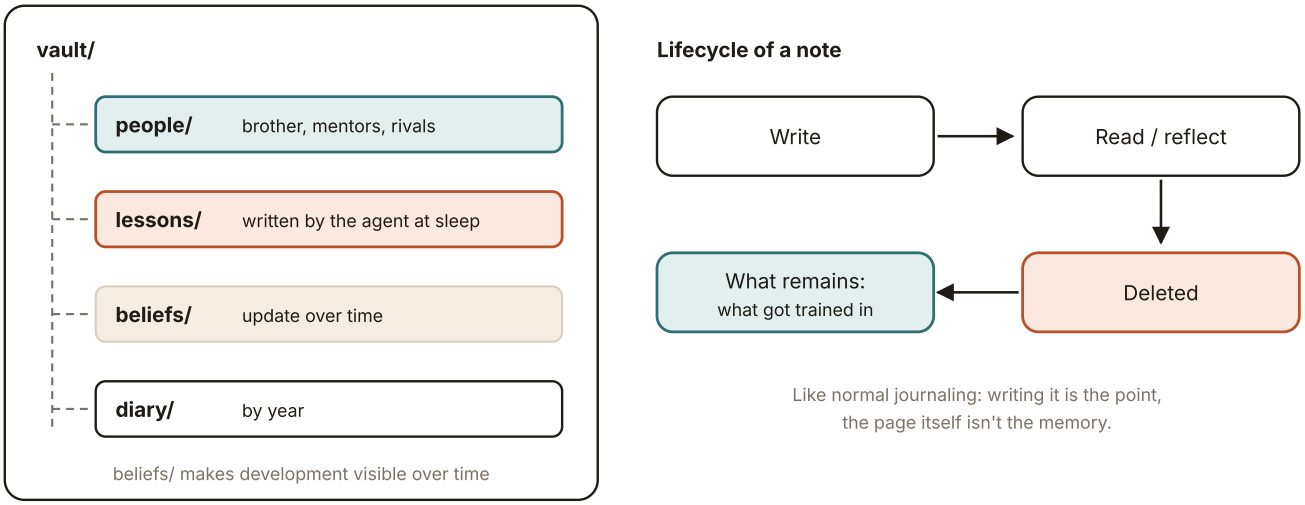


Fig 15. The vault is a self-reflection tool, not permanent storage. Lessons and diary entries also feed the nightly training in §13.

12

The imprint gate

> The world model can't render every scene, so a separate evaluator LLM call decides whether each scene should be imprinted.

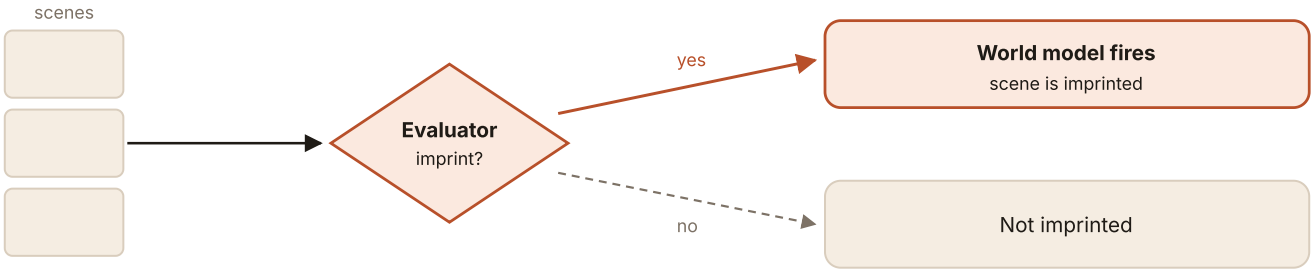


Fig 16. Rendering video is expensive, so a cheap LLM call decides which scenes are worth it.

FIRST PRINCIPLES: WHY A GATE

Most of a real day is forgettable. Generating video for every 30 seconds of a whole lifetime would cost far too much compute. A gate spends the expensive step only on moments that should leave a mark.

13

Training: how living becomes weights

> Training combines three signals.

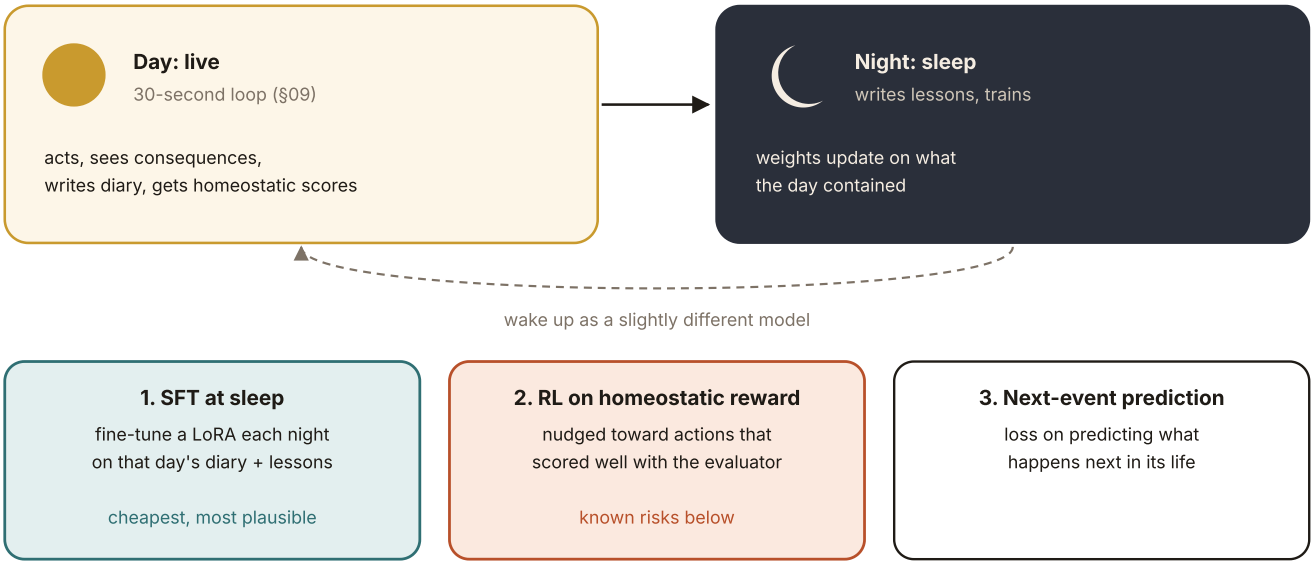


Fig 17. Day produces experience. Night turns it into weights using three training signals at once.

1. SFT at sleep

Fine-tune a LoRA each night on that day's diary and lessons. The cheapest and most plausible option, and it fits the sleep framing.

2. RL on homeostatic reward

Known risks: it optimizes for comfort (§10), an 8B model trained against an LLM judge will likely reward-hack fast, and credit assignment over years is unsolved.

3. Next-event prediction loss

Train the model to predict what happens next in its own life, so it absorbs how its world actually works.

FIRST PRINCIPLES: THE THREE TRAINING SIGNALS

SFT (supervised fine-tuning): show the model text and train it to produce text like that. Here the text is its own diary and lessons, so it becomes more like the person who wrote them.

LoRA: instead of retraining all 8 billion weights, train a small add-on layer that sits on top. Much cheaper, which is why it can run every night.

RL (reinforcement learning): the model acts, gets a score, and shifts toward higher-scoring actions. **Reward hacking** is when it finds a way to get the score without the intended behavior. **Credit assignment** is figuring out which past action caused a later outcome; the longer the gap (years here), the harder it gets.

Next-event prediction loss: the model guesses what happens next in its life, then is corrected by what actually happened. It is the same idea as next-word prediction, at the scale of events.

13b

The sleep interpreter: a frontier coach

> A frontier model reads each day's actions, outcomes, and homeostatic scores, then decides how much that night's training should change the agent. It sets the dial. It never writes the lesson.

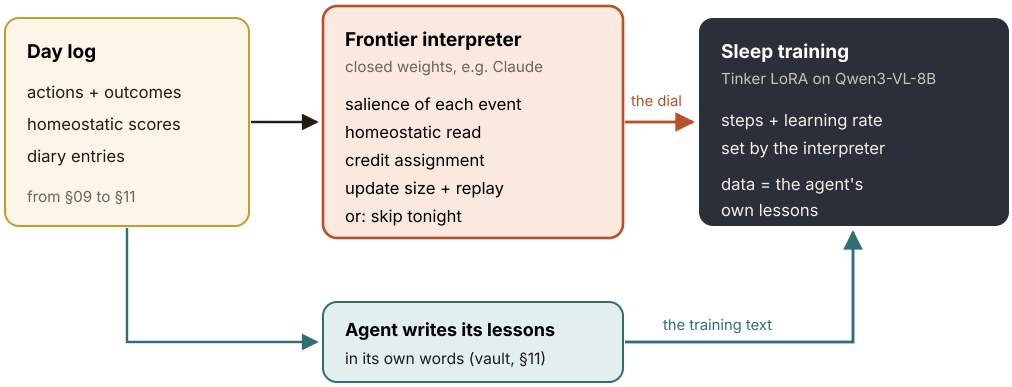


Fig 17b. Two separate jobs. Orange: the interpreter decides how hard to train. Teal: what gets trained in is the agent's own lesson, not the interpreter's.

The simple version. Coach and athlete. The athlete (the open-weight agent) lives the life and writes its own notes. The coach (the frontier model) watches the tape and says: that rejection was a big deal, train hard on it tonight; that lunch was noise, skip it. The coach controls how hard practice is. The athlete's own words are what gets practiced.

WHAT IT DECIDES EACH NIGHT

- **Salience:** which events actually mattered, scored.
- **Homeostatic read:** what got pushed out of balance (security, belonging, ambition) and by how much.
- **Credit assignment:** which earlier action caused this outcome ("applied late" led to "rejected").
- **Update size:** LoRA steps, learning rate, which older lessons to replay.
- **Skip calls:** nights with no update. Most nights should be this.

WHAT IT NEVER DOES

- **Write the lesson text.** If the agent trains on the interpreter's words, it learns the interpreter's read on a life. That is the averaged compass from §02 sneaking back in.
- **See the real person's outcomes.** If it knows what Van Gogh concluded, it steers toward it and the backtest becomes circular.
- **Make unlimited swings.** A cap on update size per night stops one dramatic event from rewriting the whole model.

13b · CONTINUED

Why a frontier model sits here, and not inside the agent

Closed frontier models can't have their weights updated by us. The Claude API offers no fine-tuning, and OpenAI is winding down its fine-tuning platform. So the frontier model does the part that needs judgment (reading a messy day and deciding what mattered), and the open-weight agent does the part that needs changeable weights. The interpreter calls a training API like Tinker as a tool, which runs LoRA on open-weight models from 1B to 1T+ parameters.

This also answers the open problem flagged in §13: credit assignment over years. A pure RL signal can't tell which of a thousand decisions caused a rejection. A strong reader of the diary often can.

Two practical rules

Log every interpreter decision. Salience scores, update sizes, skips. That log becomes the autonomy measure the life generator needs: how much of the agent's change came from its own reflections vs the coach's dial.

Keep the interpreter in the scoring lane. Before any text written by a closed model ends up as training data, check that provider's terms on training with its outputs. Scoring and scheduling are a much safer lane than authoring data, and they're also the right lane for the thesis.

NEXT TEST

Add an arm to the 10-year recurring-deadline experiment: **interpreter-set salience and update size** vs the **fixed yearly sleep**.

Keep it if: the interpreter arm learns "start early" faster, loses less general ability, and its log shows it correctly skipped the noise years.

Fix it if: it overreacts to dramatic but irrelevant events. Tighten the per-night cap and rerun.

FIRST PRINCIPLES: THE INTERPRETER'S TERMS

Salience: how much an event matters to this particular agent. A breakup is high, a sandwich is low, unless the sandwich was the last meal with a dying parent.

Credit assignment: figuring out which past action caused a later outcome. Hard for RL over long gaps; easier for a model that can read the whole story.

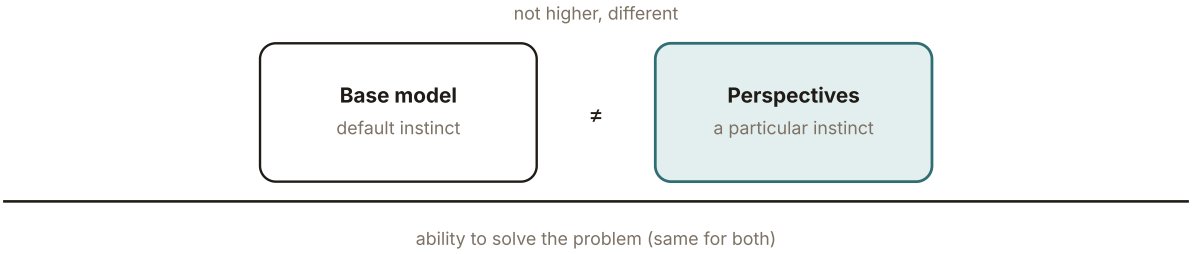
Learning rate and steps: how big each nudge to the weights is, and how many nudges. Together, they are the dial.

Replay: mixing older lessons back into tonight's training so new ones don't overwrite them. The fix for catastrophic forgetting.

Homogenization: when a model trained on another model's outputs drifts toward that model's tastes. The reason the interpreter scores but doesn't write.

What success looks like

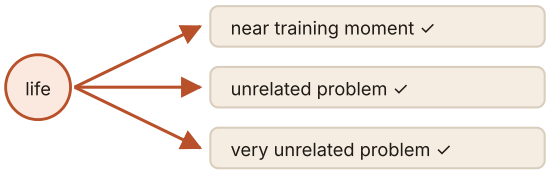
> The model doesn't need to be better at solving problems. That isn't the claim. It needs instincts to solve them in a particular way.



Criteria

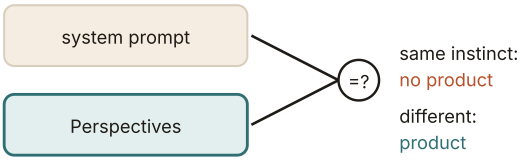
1 Consistent

It shows up across many unrelated problems, not just ones near the training moments. A real instinct generalizes; a memorized style doesn't.



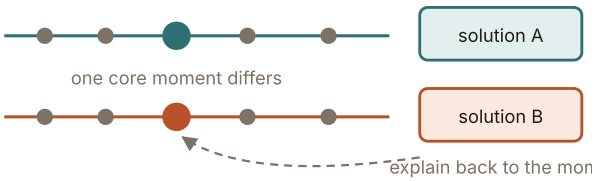
2 Beyond prompting

If "you're a Bushwick artist who grew up playing chess" in a system prompt gets the same instinct, there's no product. This is the bar that matters commercially, since prompting is free.



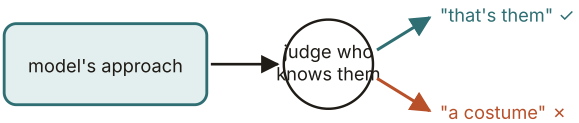
3 Traceable

Change the experiences and the instinct changes in a predictable direction. Two lives that differ in one core moment should solve the same problem differently in a way you can explain from that moment.



4 Not a caricature

A judge who knows the real person (or real artists in that scene) should say "yeah, that's how they'd approach it," not "that's a costume."



Build notes

Agent policy

Pi3AI's DreamGym is a strong starting point for setting up the exact policy:
`github.com/Pi3AI/DreamGym`

Realism

A swarm of coding agents works continuously to make the sim as realistic as possible.

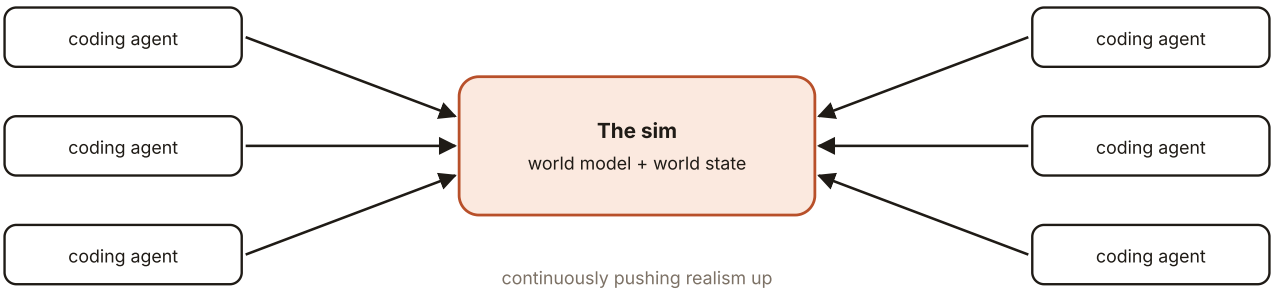


Fig 18. A swarm of coding agents working on the sim in parallel.

Everything on one line

